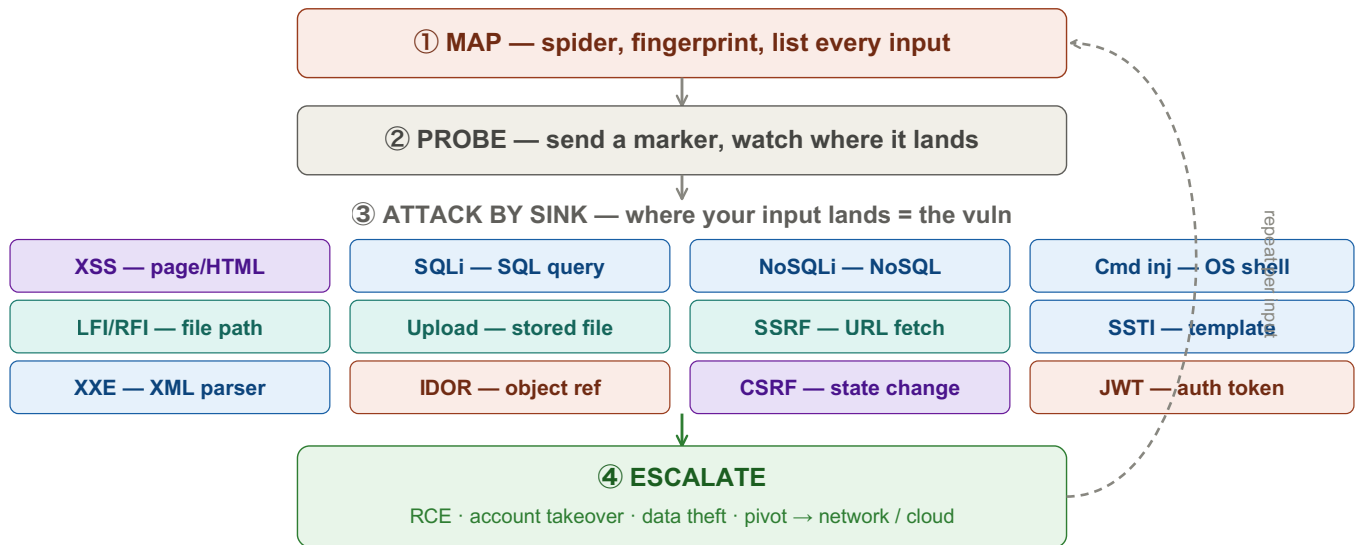


Web-App Attack Methodology

One loop: map the app → follow each input to where it lands → attack by that sink → escalate. Almost every web vuln is "my input reached a dangerous place unsanitised." This page picks which cheatsheet to open.



Part 1 — The workflow (at a glance)

The four phases

Loop

- ① MAP spider the app, fingerprint the stack, list every input surface
- ② PROBE send a marker/metachar to each input; watch where it lands + what breaks
- ③ ATTACK the sink your input reaches decides the vuln → open that cheatsheet
- ④ ESCALATE turn the bug into RCE / account takeover / data theft / pivot → repeat per input

Decision: "where does my input go?" → which attack

Your input lands in...	Symptom	Attack → sheet
HTML / the page	renders as markup	XSS
A SQL query	quote breaks it, DB errors	SQLi (+ sqli-syntax)
A NoSQL query	operator/JSON injection works	NoSQL injection
An OS shell command	metachars run commands	Command injection
A file path (include)	traversal reads files	LFI / RFI
A server-side URL fetch	server requests your URL	SSRF
A template engine	<code>{{7*7}}</code> → 49	SSTI
An XML parser	entities are resolved	XXE
An object ID / reference	changing it exposes others' data	IDOR / access control
A state-changing request	no anti-CSRF token	CSRF
An auth token	JWT alg/signature weak	JWT attacks
An uploaded file	server stores + executes it	File upload bypass

The tool through all of it: **Burp Suite** — proxy everything, then Repeater (hand-test) + Intruder (fuzz/brute). See [burp-suite/](#).

Part 2 — Per-phase detail

① Map the application

Recon

- ☐ Proxy the whole app through Burp; browse every feature (build the site map)
- ☐ Fingerprint: server, framework, language (Wappalyzer, headers, error pages, cookies)
- ☐ Enumerate content: dirs/files/params/vhosts → 01-recon/web-enumeration (ffuf/feroxbuster)
- ☐ List every input: query params, path segments, POST (form/JSON/multipart), headers (Cookie, User-Agent, Referer, X-Forwarded-For, Host), uploads, GraphQL/REST
- ☐ Note auth boundaries: which pages need a session, which roles (user vs admin)

② Probe each input (one polyglot marker)

Probe

```
test'"<x>{{7*7}}${7*7}|id;sleep0
```

```
" → SQL error / broken HTML → SQLi / XSS
<x> renders as a tag → XSS
{{7*7}} or ${7*7} → 49 → SSTI
reflected in a server-side fetch → SSRF
path/file behaviour changes → LFI / RFI
timing/OOB on ; | ` → command injection
```

③ Attack by sink · ④ Escalate

Open the matching sheet (Part 1 table). Each follows the same shape: **Overview** → **Detailed walkthrough** → **Payload arsenal**. Then turn the win into impact:

Escalate

```
XSS → steal session / act as victim / account takeover
SQLi → dump creds → crack (hashcat/john) → reuse; or --os-shell → RCE
Cmd injection → reverse shell → local privesc (linux/windows-privesc)
LFI/RFI, upload, SSTI, XXE → RCE → foothold
SSRF → cloud metadata → IAM creds → cloud account (azure/entra sheets)
IDOR / JWT / auth → horizontal/vertical access → admin → takeover
pivot: looted creds → other services · foothold → 06-network · cloud creds → 07-cloud
```

Part 3 — Quick reference

Coverage map (this folder)

Class	Sheet
Injection — SQL	sqli, sqli-syntax
Injection — NoSQL	nosql-injection
Injection — OS command	command-injection
Injection — template	ssti
Injection — XML	xxe
Client-side	xss, csrf
File / path	lfi-rfi, file-upload-bypass
Server-side request	ssrf
Access control	idor-access-control, jwt-attacks
Tooling	burp-suite/

OWASP Top 10 (2021) → sheets

A01 Broken Access Control → idor-access-control, jwt-attacks, csrf
A03 Injection → sql, xss, command-injection, nosql-injection, ssti
A05 Security Misconfiguration → file-upload-bypass, xxe
A10 Server-Side Request Forgery → ssrf

Key idea: every web bug is the same story — untrusted input reaches a place that trusts it (a query, a shell, a parser, a file path, a URL, a token). Map the app, follow each input to its sink, attack by that sink, then escalate. This page picks the door; the individual cheatsheets kick it in.